```
$xpath->registerNamespace('foo', 'urn:foo.ns');
$xpath->registerPhpFunctionNS(
    'urn:foo.ns',
    'someFunction',
    function (array $nodes) {
        // do something with $nodes here...
    }
);
$xpath->query('//ns:body[foo:someFunction(.)]');
```

querySelector() (Because XPath Looks Hard)

I've gone (fairly) deep on XPath because I'm opinionated, and my principle reason was that XPath gives you more tools for DOM traversal than CSS selectors; there are tricks you can pull with XPath that just don't exist with CSS selectors. With that said, though, I would be doing you a disservice if I didn't also show a couple of alternatives. I mentioned earlier in this section that PHP 8.4 introduces support for using selector expressions for DOM traversal. Specifically, this is via `Dom\HTMLDocument` and `DOM\HTMLElement` – the HTML5-compliant replacements for `DOMDocument` and `DOMElement`. These new classes provide a couple of methods that have been "borrowed" from JavaScript's own DOM API, named `querySelector()` and `querySelectorAll()`. (These new methods exist on the `Dom\Document` and `Dom\Element` parent classes, so you can use them with HTML or XML).

CSS (Cascading Style Sheets) is a declarative language used to describe how an HTML document should be presented (rendered) by a client. The most fundamental unit of CSS is the ruleset, which is formed from selector expressions followed by declaration blocks. The declarations in each block are applied to the elements that match the selector; the general syntax is as follows:

```
<selector_expression> {
    <declaration>;
    <declaration>;
}
```